

UNIVERSITÉ CHEIKH HAMIDOU KANE
(Ex Université Virtuelle du Sénégal — UVS)
Master Ingénierie Logicielle — Promotion 8

RAPPORT DE PROJET

Technologie d'Application Web

Conception et développement d'une plateforme de gestion universitaire

Champ	Information
Étudiant	Boune Abdoulaye Mbacke
INE	N00398120201
Email	bouneabdoulaye.mbacke@unchk.edu.sn
Filière	Master Ingénierie Logicielle — P8
Année universitaire	2025 – 2026
Module	Technologie d'Application Web
Dépôt GitHub	github.com/abdallah020/Projet-TECHNOLOGIE-D-APPLICATION-WEB

Sommaire

Sommaire	2
1. Contexte du projet	4
2. Objectifs du projet.....	4
2.1 Objectifs spécifiques.....	4
3. Architecture du système	6
3.1 Vue d'ensemble.....	6
3.2 Couche présentation (Frontend).....	6
3.3 Couche API / Middleware.....	6
3.4 Couche persistance (Backend / Base de données).....	6
4. Modules fonctionnels.....	8
4.1 Module Authentification	8
4.2 Module Formation.....	8
4.3 Module Inscriptions.....	8
4.4 Module Étudiants	8
4.5 Module Enseignants	8
4.6 Module Administration	8
4.7 Synthèse des modules.....	9
5. Gestion des rôles et dashboards	10
5.1 Tableau comparatif des rôles	10
5.2 Dashboard Administrateur	10
5.3 Dashboard Étudiant.....	10
5.4 Dashboard Enseignant.....	10
6. Base de données	11
6.1 Tables principales	11
6.2 Relations principales	11
6.3 Schéma relationnel simplifié	11
7. Technologies utilisées	12
7.1 Frontend.....	12
7.2 Backend	12
7.3 Base de données et outils.....	12
8. Sécurité de l'application	13
9. Installation et exécution.....	14
9.1 Prérequis	14

9.2 Backend (Spring Boot)	14
9.3 Frontend (Angular)	14
9.4 Base de données PostgreSQL	14
10. Fonctionnalités principales.....	15
11. Organisation du projet	15
11.1 Référentiel GitHub.....	15
12. Difficultés rencontrées et perspectives d'évolution.....	16
12.1 Difficultés rencontrées	16
12.2 Perspectives d'évolution	16
13. Conclusion	17

1. Contexte du projet

Dans le cadre du cours de Technologie d'Application Web du Master Ingénierie Logicielle (Promotion 8) à l'Université Cheikh Hamidou Kane (ex UVS), il a été demandé de concevoir et développer une application web complète dédiée à la gestion universitaire.

Le projet consiste à mettre en place une plateforme numérique pour l'Université Cheikh Hamidou Kane permettant de centraliser et de simplifier la gestion des étudiants, des enseignants, des formations et des activités administratives et pédagogiques de l'établissement. Cette plateforme s'inscrit dans une logique de digitalisation des processus universitaires, en remplaçant des procédures manuelles ou dispersées par un système unifié, sécurisé et accessible à distance.

Périmètre du document

Ce rapport décrit l'ensemble du cycle de conception du projet : contexte, objectifs, architecture technique, modélisation des données, modules fonctionnels, gestion des rôles, sécurité, déploiement et perspectives d'évolution.

2. Objectifs du projet

L'objectif principal est de concevoir une application web modulaire, sécurisée et évolutive, capable de répondre aux besoins de gestion d'un établissement universitaire moderne. Plus précisément, le projet vise à :

- Gérer les utilisateurs de la plateforme (étudiants, enseignants, administrateurs) avec un contrôle d'accès basé sur les rôles ;
- Gérer les formations proposées par l'université ainsi que les inscriptions des étudiants à ces formations ;
- Permettre la validation ou le rejet des demandes d'inscription par les administrateurs ;
- Centraliser les données pédagogiques et administratives au sein d'une base de données relationnelle unique ;
- Mettre en place une architecture moderne en trois couches (frontend, API, backend) garantissant la séparation des responsabilités, la maintenabilité et la sécurité du système.

2.1 Objectifs spécifiques

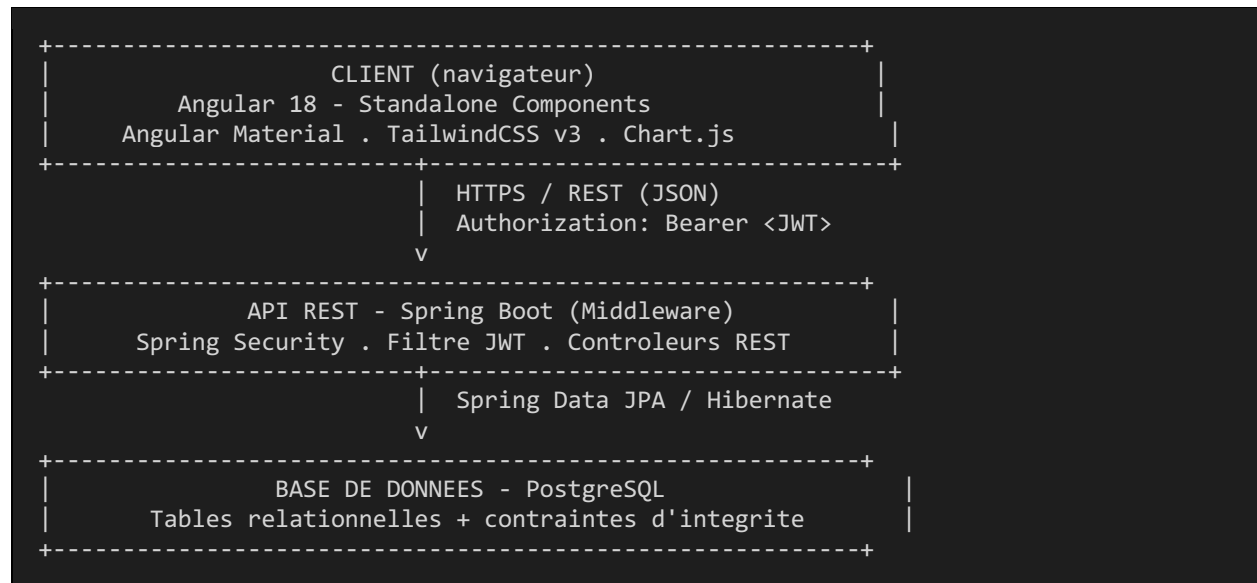
Axe	Objectif spécifique
Authentification	Sécuriser l'accès via JWT et empêcher tout accès non autorisé aux ressources
Gestion académique	Offrir un suivi clair du parcours de l'étudiant, de l'inscription à l'évaluation

Axe	Objectif spécifique
Administration	Donner à l'administrateur une vue d'ensemble et des outils de pilotage (statistiques, validation)
Expérience utilisateur	Proposer une interface claire, réactive et adaptée à chaque rôle (dashboard dédié)
Qualité technique	Garantir une architecture découplée, testable et déployable indépendamment (frontend / backend)

3. Architecture du système

L'application repose sur une architecture en trois couches (3-tier architecture), garantissant une séparation nette entre la présentation, la logique métier et la persistance des données. Cette approche facilite la maintenance, les tests et l'évolutivité du système.

3.1 Vue d'ensemble



3.2 Couche présentation (Frontend)

Le frontend est développé en Angular 18 selon une architecture standalone (sans NgModules), ce qui simplifie l'organisation du code et améliore les temps de compilation. Il est responsable de :

- L'affichage des tableaux de bord (dashboards) spécifiques à chaque rôle ;
- La saisie et la validation des formulaires (inscriptions, création de formations, etc.) ;
- La consultation des données via des tableaux dynamiques avec filtres et recherche ;
- La visualisation de statistiques grâce à des graphiques générés avec Chart.js ;
- La gestion de la session utilisateur (stockage du token JWT, garde de routes / route guards).

3.3 Couche API / Middleware

L'API REST, développée avec Spring Boot, joue le rôle d'intermédiaire entre le frontend et la base de données. Elle expose des endpoints REST structurés par ressource (étudiants, enseignants, formations, inscriptions, cours, communications) et applique les règles de sécurité via Spring Security et un filtre d'authentification JWT.

3.4 Couche persistance (Backend / Base de données)

La persistance des données est assurée par PostgreSQL, avec Hibernate/JPA comme couche d'abstraction objet-relationnel (ORM). Cette couche porte la logique métier (services), la validation des règles de gestion et la garantie de l'intégrité référentielle entre les entités.

4. Modules fonctionnels

L'application est découpée en six modules fonctionnels principaux, chacun correspondant à un domaine métier de la gestion universitaire.

4.1 Module Authentification

- Inscription d'un nouvel utilisateur (selon le rôle autorisé) ;
- Connexion sécurisée par identifiants avec génération d'un jeton JWT ;
- Gestion des rôles et redirection automatique vers le dashboard correspondant.

4.2 Module Formation

- Création d'une formation par l'administrateur (intitulé, description, capacité, dates) ;
- Consultation de la liste des formations disponibles ;
- Gestion du cycle de vie d'une formation via un statut : PLANIFIÉE, ACTIVE, TERMINÉE.

4.3 Module Inscriptions

- Soumission d'une demande d'inscription par l'étudiant à une formation ;
- Validation ou rejet de la demande par l'administrateur ;
- Suivi en temps réel du statut de la demande (EN_ATTENTE, VALIDÉE, REJETÉE).

4.4 Module Étudiants

- Gestion du profil et des informations personnelles de l'étudiant ;
- Suivi des inscriptions actives et passées ;
- Consultation de l'historique académique.

4.5 Module Enseignants

- Consultation des formations et cours assignés ;
- Suivi pédagogique des étudiants inscrits ;
- Gestion des activités liées aux cours (contenus, communications).

4.6 Module Administration

- Supervision globale du système et des utilisateurs ;
- Validation ou rejet centralisé des demandes d'inscription ;
- Gestion des formations, enseignants et étudiants ;
- Accès à des statistiques globales (effectifs, taux de validation, répartition par formation).

4.7 Synthèse des modules

Module	Acteur(s) principal(aux)	Fonctions clés
Authentification	Tous	Connexion, inscription, gestion des rôles, JWT
Formations	Admin, Enseignant, Étudiant	Création, consultation, statuts (Planifiée/Active/Terminée)
Inscriptions	Étudiant, Admin	Demande, validation, rejet, suivi de statut
Étudiants	Étudiant, Admin	Profil, historique, suivi académique
Enseignants	Enseignant, Admin	Formations assignées, suivi pédagogique
Administration	Admin	Supervision, statistiques, gestion globale

5. Gestion des rôles et dashboards

L'application repose sur un contrôle d'accès basé sur les rôles (RBAC — Role-Based Access Control), garantissant que chaque utilisateur n'accède qu'aux fonctionnalités et données correspondant à son profil. Trois rôles sont définis : ADMIN, ENSEIGNANT et ÉTUDIANT.

5.1 Tableau comparatif des rôles

Fonctionnalité	Admin	Enseignant	Étudiant
Gestion des utilisateurs	Oui	Non	Non
Création de formations	Oui	Non	Non
Validation des inscriptions	Oui	Non	Non
Consultation des formations assignées	Oui (toutes)	Oui (les siennes)	Oui (publiques)
Demande d'inscription	Non applicable	Non applicable	Oui
Suivi pédagogique	Vue globale	Oui	Non
Statistiques globales	Oui	Non	Non
Communications	Émission globale	Émission ciblée	Réception

5.2 Dashboard Administrateur

- Gestion complète du système et des comptes utilisateurs ;
- Validation ou rejet des demandes d'inscription ;
- Gestion des formations (création, modification, changement de statut) ;
- Accès aux statistiques globales (graphiques Chart.js : effectifs, répartition par formation, taux de validation).

5.3 Dashboard Étudiant

- Inscription à une ou plusieurs formations ;
- Suivi du statut de ses inscriptions ;
- Consultation et mise à jour de ses informations personnelles.

5.4 Dashboard Enseignant

- Accès aux formations qui lui sont assignées ;
- Suivi pédagogique des étudiants inscrits à ses cours ;
- Gestion des cours et des communications associées.

6. Base de données

La persistance des données repose sur une base de données relationnelle PostgreSQL. Le modèle de données est organisé autour des entités centrales du domaine universitaire, reliées entre elles par des clés étrangères garantissant l'intégrité référentielle.

6.1 Tables principales

Table	Rôle
utilisateurs	Compte de connexion (identifiants, rôle, statut) commun à tous les profils
etudiants	Informations spécifiques aux étudiants, liées à un compte utilisateur
enseignants	Informations spécifiques aux enseignants, liées à un compte utilisateur
formations	Catalogue des formations proposées (intitulé, statut, capacité)
inscriptions	Demandes d'inscription des étudiants aux formations, avec statut
cours	Cours rattachés à une formation et à un enseignant
emplois_du_temps	Planification des séances de cours
reunions	Réunions pédagogiques ou administratives
budgets	Suivi budgétaire lié aux formations ou à l'administration

6.2 Relations principales

- Un étudiant peut soumettre plusieurs demandes d'inscription (relation 1–N entre `etudiants` et `inscriptions`) ;
- Une formation peut recevoir plusieurs inscriptions (relation 1–N entre `formations` et `inscriptions`) ;
- Une formation peut comporter plusieurs cours, chacun rattaché à un enseignant (relations 1–N) ;
- Des contraintes de clés étrangères assurent l'intégrité référentielle entre toutes les tables liées.

6.3 Schéma relationnel simplifié

```
utilisateurs (1)---(1) etudiants
      |
      +---(1)---(1) enseignants

formations (1)---(N) inscriptions (N)---(1) etudiants
      |
      +---(1)---(N) cours (N)---(1) enseignants
                  |
                  +---(1)---(N) emplois_du_temps
```

7. Technologies utilisées

7.1 Frontend

Technologie	Usage
Angular 18	Framework principal, architecture standalone components
TypeScript	Langage de développement typé
TailwindCSS v3	Mise en forme utilitaire et responsive design
Angular Material	Composants UI (tableaux, formulaires, dialogues)
Chart.js	Visualisation de données statistiques sur les dashboards
RxJS	Gestion de la programmation réactive et des flux asynchrones

7.2 Backend

Technologie	Usage
Spring Boot	Framework principal de l'API REST
Spring Security	Sécurisation des endpoints et gestion des autorisations
JWT (JSON Web Token)	Authentification stateless entre client et serveur
Hibernate / JPA	Mapping objet-relationnel (ORM) vers PostgreSQL

7.3 Base de données et outils

Catégorie	Outil
Base de données	PostgreSQL
Gestion de versions	Git & GitHub
Tests d'API	Postman
Environnements de développement	VS Code (frontend) / IntelliJ IDEA (backend)

8. Sécurité de l'application

La sécurité est un axe central du projet, étant donné la sensibilité des données académiques et personnelles manipulées. Plusieurs mécanismes sont mis en œuvre :

- Authentification stateless via JWT : chaque requête authentifiée porte un jeton signé, évitant le stockage de session côté serveur ;
- Hachage des mots de passe avant persistance (aucun mot de passe n'est stocké en clair) ;
- Autorisations basées sur les rôles (RBAC) appliquées à la fois côté API (Spring Security) et côté interface (guards Angular) ;
- Validation des données en entrée, côté client et côté serveur, afin de limiter les risques d'injection ou de données incohérentes ;
- Communication via HTTPS recommandée en production pour chiffrer les échanges client-serveur.

Bonne pratique

Le jeton JWT ne doit jamais être stocké dans le localStorage sans précaution sur des applications exposées au risque XSS ; une attention particulière a été portée à la durée de vie du token et à son renouvellement.

9. Installation et exécution

9.1 Prérequis

- Node.js (version 18 ou supérieure) et npm pour le frontend Angular ;
- JDK 17 ou supérieur et Maven pour le backend Spring Boot ;
- PostgreSQL installé et configuré localement ou accessible à distance ;
- Git pour le clonage du dépôt.

9.2 Backend (Spring Boot)

```
cd backend
mvn clean install
mvn spring-boot:run
```

Le backend sera accessible sur :

```
http://localhost:8080
```

9.3 Frontend (Angular)

```
cd frontend
npm install
ng serve
```

Le frontend sera accessible sur :

```
http://localhost:4200
```

9.4 Base de données PostgreSQL

Création de la base :

```
CREATE DATABASE universite_db;
```

Importation du dump :

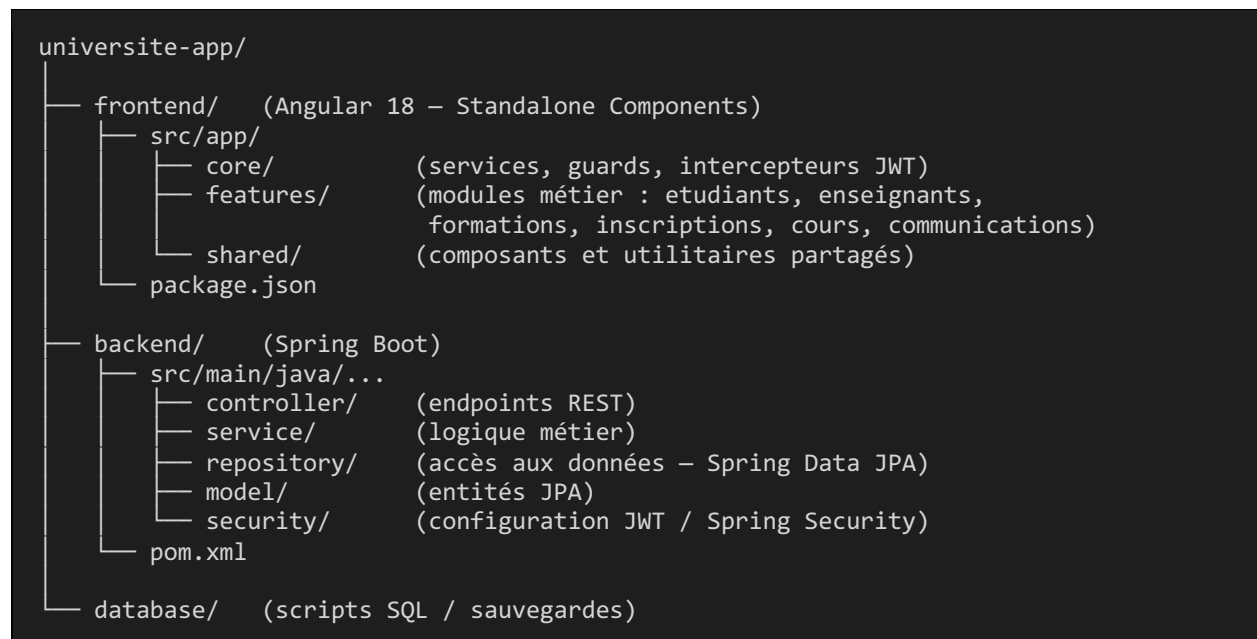
```
pg_restore -U postgres -d universite_db universite_db.backup
```

10. Fonctionnalités principales

- Authentification sécurisée par JWT ;
- Gestion des rôles utilisateurs (ADMIN, ENSEIGNANT, ÉTUDIANT) ;
- Dashboard personnalisé par profil utilisateur ;
- Gestion complète des inscriptions, de la demande à la validation ;
- Validation administrative centralisée ;
- Filtrage et recherche dynamique sur les tableaux de données ;
- API REST structurée et documentée par ressource métier.

11. Organisation du projet

L'arborescence générale du dépôt est structurée comme suit :



11.1 Référentiel GitHub

Le projet est disponible publiquement à l'adresse suivante : github.com/abdallah020/Projet-TECHNOLOGIE-D-APPLICATION-WEB

12. Difficultés rencontrées et perspectives d'évolution

12.1 Difficultés rencontrées

- Synchronisation des statuts entre les différentes entités liées (inscriptions, formations) afin d'éviter les incohérences ;
- Mise en place fine des autorisations par rôle, à la fois côté frontend (guards, affichage conditionnel) et côté backend (Spring Security) ;
- Gestion du cycle de vie du token JWT (expiration, renouvellement, déconnexion automatique).

12.2 Perspectives d'évolution

- Mise en place de notifications en temps réel (WebSocket) pour les validations d'inscription ;
- Ajout d'un module de gestion des notes et des évaluations ;
- Génération automatisée de documents administratifs (certificats de scolarité, relevés de notes) ;
- Mise en place de tests automatisés (unitaires et end-to-end) pour fiabiliser les évolutions futures ;
- Déploiement conteneurisé (Docker) pour faciliter la mise en production.

13. Conclusion

Ce projet a permis de mettre en pratique une architecture moderne basée sur Angular et Spring Boot, dans le cadre de la gestion numérique d'un établissement universitaire. Il démontre la capacité à concevoir une application complète, intégrant la gestion des utilisateurs, des rôles, une API REST sécurisée et une base de données relationnelle structurée.

Le système implémente une gestion des rôles (RBAC) assurant une séparation claire entre les administrateurs, les enseignants et les étudiants, tout en offrant à chaque profil une expérience adaptée à ses besoins. Au-delà de l'aspect technique, ce travail illustre une démarche d'ingénierie logicielle complète : analyse des besoins, conception d'architecture, modélisation des données, implémentation sécurisée et perspectives d'amélioration continue.